


AMiT
Arba Minch Institute of Technology
አርባ ምንጭ ቴክኖሎጂ ኢንስቲትዩት
Arba Minch University
አርባ ምንጭ ዩኒቨርሲቲ

Faculty of Computing and Software Engineering

Department of Software Engineering

# SENG5232- Software Architecture and Design

# Chapter 2: Design Patterns

*"The best designers will use many **design patterns** that dovetail and intertwine to produce a greater whole."*

*Erich Gamma*

By Aliazar D.(MSc in Software Eng.) Email :- aliazar.deneke@amu.edu.et

# Contents

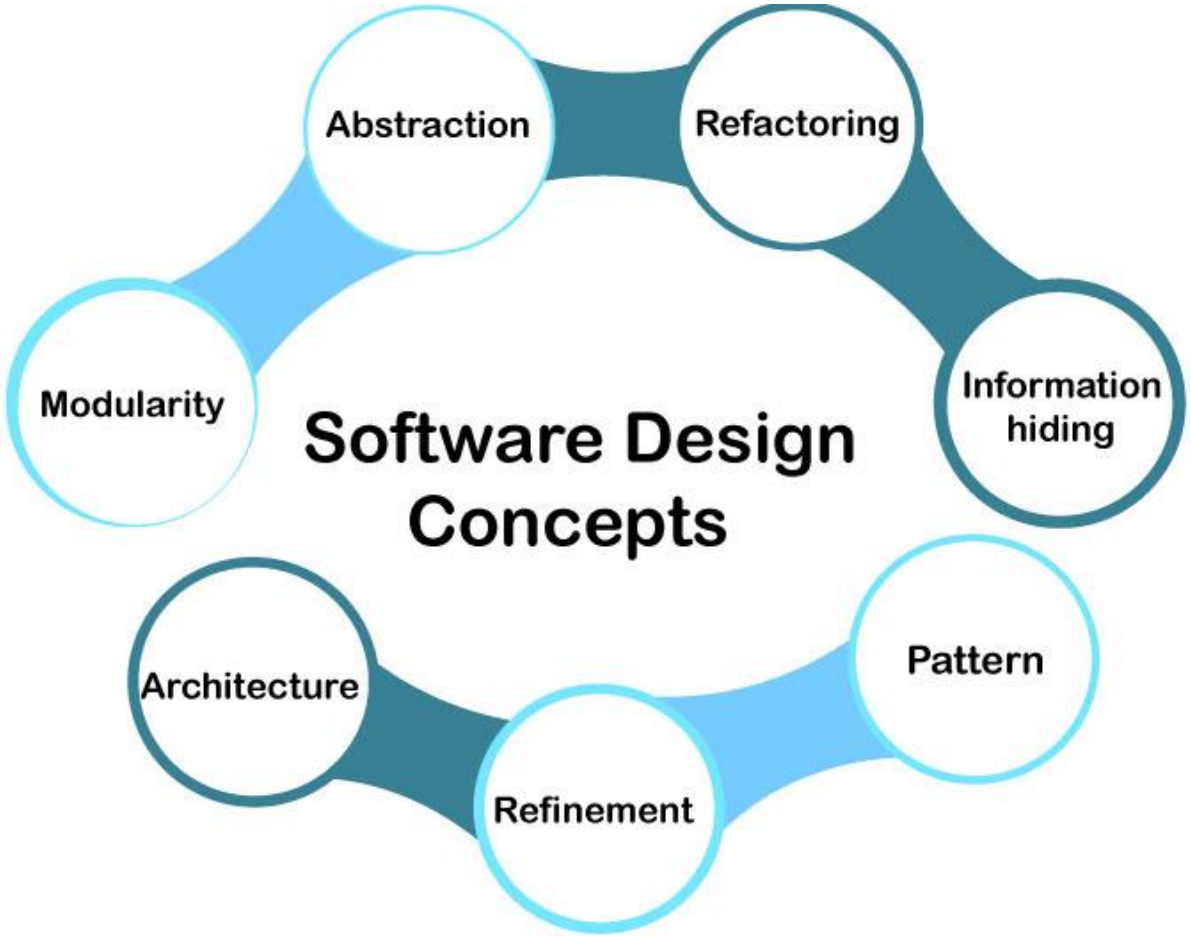
- 1 Software Design Concepts
- 2 Object Oriented Concepts
- 3 Key Architectural Principles
- 4 OO Design Principles
- 5 Pattern Oriented Design

# Brainstorming Questions

**BS1**: What is OO programming and can you define or give an example ?

**BS2**: Why is object-oriented programming important for software design ?

# Overview of Design Concepts/#1




**1. Modularity:** The practice of dividing a complex system into smaller, independent, and interchangeable components that can be developed and maintained separately.

**2. Abstraction:** The process of simplifying complex systems by focusing on essential characteristics and ignoring unnecessary details.

**3. Refactoring:** The systematic process of restructuring existing code to improve its readability, maintainability, and efficiency without changing its external behavior.

**4. Information hiding:** The principle of encapsulating the internal details of a module or component to prevent direct access and manipulation, promoting data security and reducing dependencies.

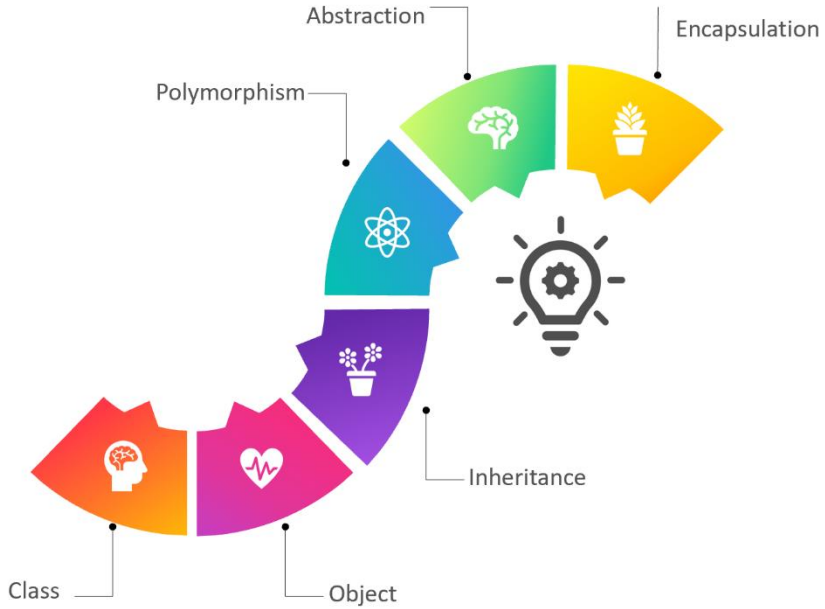
**5. Architecture:** The high-level structure and organization of a system or application, including the components, their interactions, and the design principles that guide their development.

**6. Refinement:** The iterative process of gradually elaborating the design and implementation of a system, starting with a high-level concept and progressing to more detailed and concrete levels.

**7. Pattern:** A recurring and proven solution to a common problem in software design, often documented as a best practice or template for solving specific types of issues.

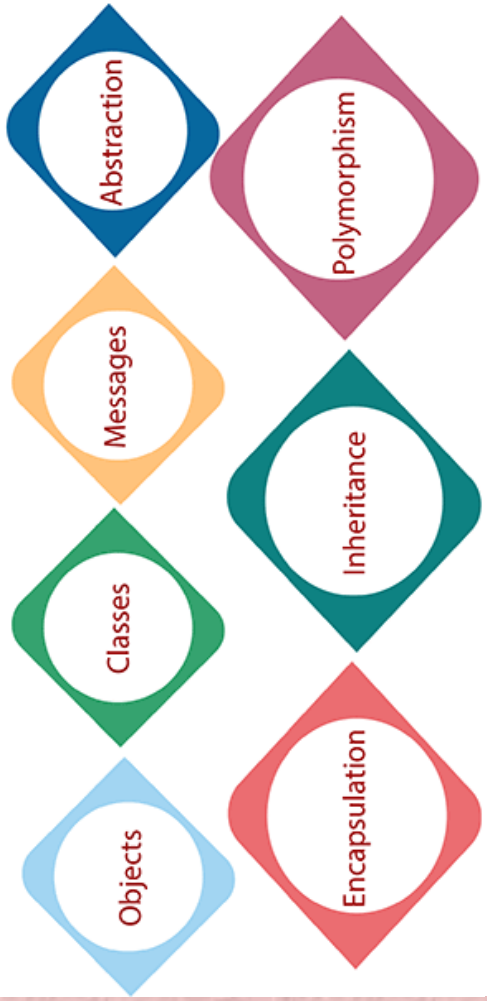
# Overview of OO concepts/#2

- The history of object-oriented programming (OOP) dates back to the 1960s when foundational concepts like objects, classes, and inheritance hierarchies were first developed.
- The Simula language, created by the Norwegian Computing Center, played a pivotal role in introducing these concepts.
- In the 1970s, Alan Kay's work at Xerox PARC led to the development of the Smalltalk language, further advancing OOP principles.
- These milestones laid the groundwork for modern OOP languages and methodologies, shaping the way software is designed and organized today.




- *Since then, OO concepts are considered important in software development because they address fundamental issues of adaptation and evolution.*
- ***Adaptation** is the system's capacity to adjust to changing conditions or requirements, while **evolution** involves the ongoing refinement and growth of the software over time.*

# Overview of OO concepts/#2




- **Classes:** A class is a generalized description of an object. An object is an instance of a class.
  - It defines all the attributes, which an object can have and methods, which represents the functionality of the object.
- **Objects**: All entities involved in the solution design are known as objects.
  - It can be physical or conceptual and groups which can contain *data* and *code*:
    - *Data* in the form of fields (often known as attributes or properties), and
    - *Code* in the form of procedures (often known as methods or operations).
      - Some approaches, refer to the *operation* as the *specification* of a function performed by an object and the *method* as the *implementation* of the function.
  - The signature of an *operation* specifies the *operation's name, parameters, and return value*.
  - An object's *interface* is the set of operations it provides, as defined by the signatures of the operations.
  - An object's *type* is defined by its interface, *implementation* is defined by its class.
- **Messages:** Objects communicate by message passing.

# Overview of OO concepts/#2

**The Four Pillars of Object Oriented Programming**

- **Abstraction** In object-oriented design, complexity is handled using abstraction.
  - It is the removal of the irrelevant and the amplification of the essentials. i.e. a real-life example of a ***man driving a car.***
- **Encapsulation:** is also called an information hiding concept.
  - The data and operations are linked to a single unit.
  - It is not only bundles essential information of an object together but also restricts access to the data and methods from the outside world.
  - A real-life example in a ***company***, there are different sections like the accounts section, finance section, sales section
- **Inheritance:** OOD allows similar classes to stack up in a hierarchical manner where the lower or sub-classes can import, implement, and re-use allowed variables and functions from their immediate super classes.
- **Polymorphism**: OO languages provide a mechanism where methods performing similar tasks but vary in arguments, can be assigned the same name.

# Overview of OO concepts/#2

```
abstract class Animal {
   abstract void soundOfAnimal();  // It is just an idea
}
class Cat extends Animal {
   void soundOfAnimal() {
      System.out.println("Meoh");
      //Implementation of the idea according to requirements of sub class
   }
}
class Dog extends Animal{
   void soundOfAnimal() {
      System.out.println("Bow Bow");
      //Implementation of the idea according to requirements of sub class
   }
}
```

*Examples of Abstract implementation in Java*

# Overview of OO concepts/#2

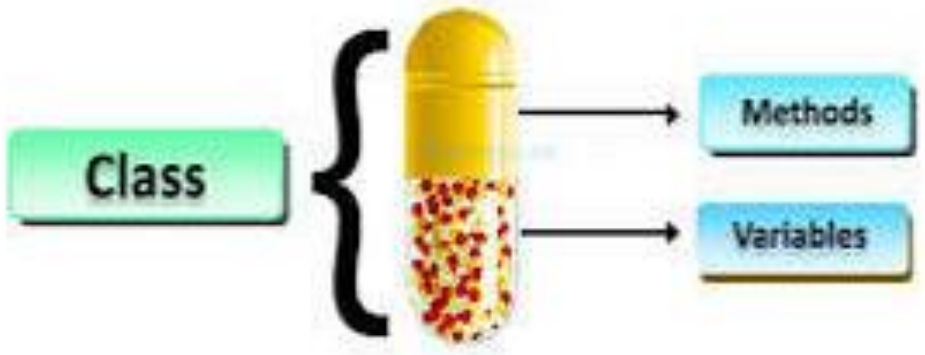
† *Encapsulation (Information hiding)* is a fundamental software design concept relevant to the design of all software.

- » Early systems were *error-prone* and *difficult* to modify because of the widespread of *global data*.
- » *David Parnas* advocated information hiding as a criterion for *decomposing* a software system into *modules*.
- » Each *information hiding* module should hide a design decision that is considered likely to change.
- » Each *changeable decision* is called the *secret* of the module.

# Overview of OO concepts/#2

- With *information hiding*, the information that could potentially change is *encapsulated* (i.e., hidden) inside an object.
- The information can be *externally accessed* only indirectly by the *invocation of operations*.
- Thus, the *hidden information* and the *operations* that access it are bound together to form an *information hiding object*.
- Important ***data structures*** can be accessed by several objects and without ***information hiding***, any change to the data structure is likely to require changes to all the objects.
- This form of information hiding is called ***data abstraction***.

# Overview of OO concepts/#2




*Example of global access to Variable(functional solution)*



School Bag




Bank

Real life Example

# Overview of OO concepts/#2

† The design of an object (or class) is *two-step* process:

» First to design the *interface*, which is external view

» Then to design the *internals*

※ The first is part of *high-level design*

※ The second step is part of the *detailed design*

† This is an *iterative process* to decide what should be *externally visible* and what *should not*.

# Overview of OO concepts/#2

† *Inheritance* is a mechanism for *sharing* and *reusing* code between classes.

» A child class inherits the *properties* of a parent class.

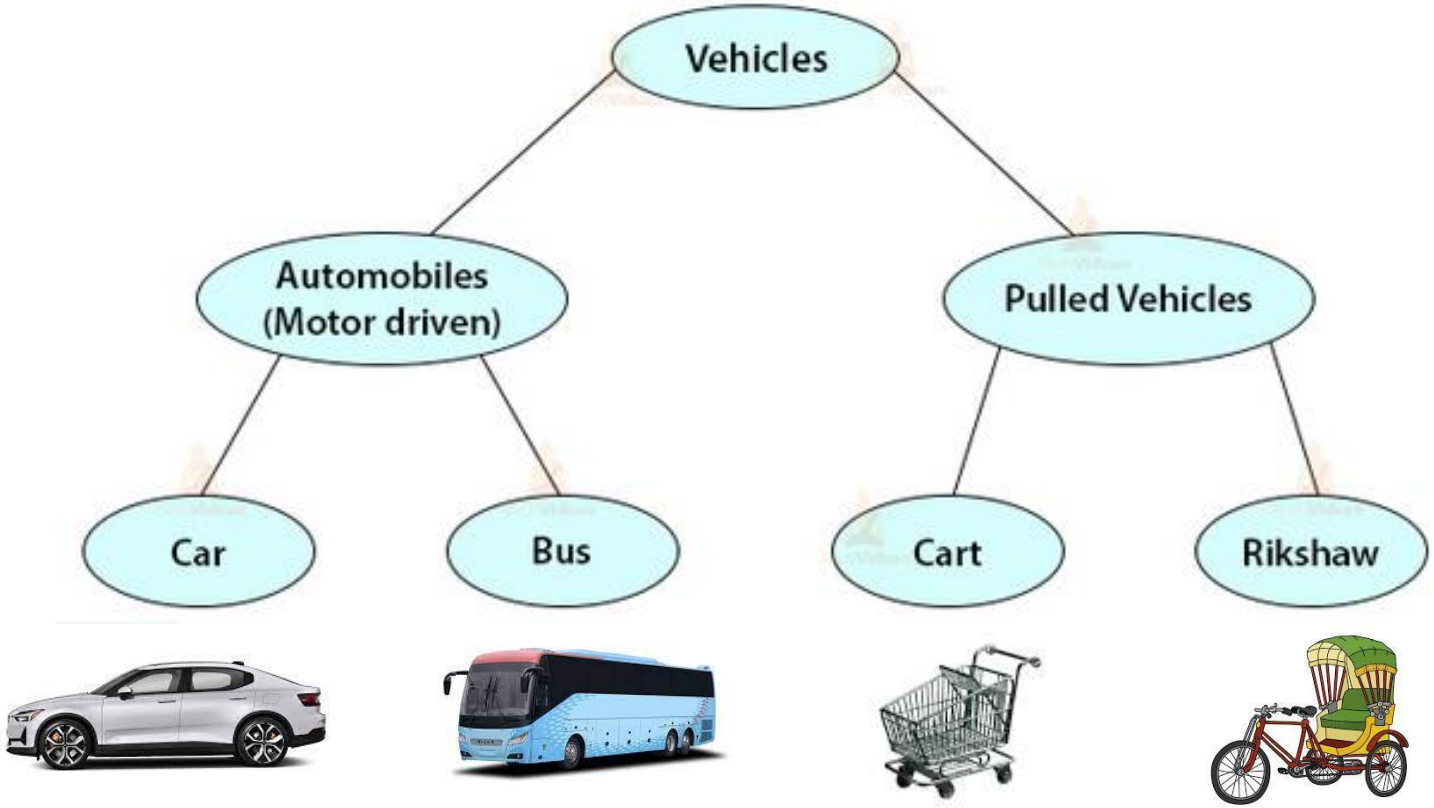
» The parent class is referred to as a *super class* or *base class*.

» The child class is referred to as a *subclass* or *derived class.*

» The child adapts the operations by adding *new operations* or by *redefining* existing operations.

» It is also possible for a child class to *override an operation* of the parent; though, such overriding is not *recommended because it can lead to unexpected behavior and code that is harder to maintain.*

# Overview of OO concepts/#2




*Example of inheritance class*

# Key Architecture Principles/#3

» Build to Change Instead of Building to Last.

» Reduce Risk and Model to Analyze.

» Use Models and Visualizations as a Communication and Collaboration Tool.

» Use an Incremental and Iterative Approach.

# Key OO Design Principles/#4

**Characteristics of a bad design**

- ※ **Rigidity** - every change affects too many other parts of the system.
- ※ **Fragility** - a change causes unexpected parts of the system break .
- ※ **Immobility** - hard to reuse in another application.

The OO design principles that we will discuss lay the foundation of solid OO practices

- » Open-Closed principle
- » Design by contract/Liskov substitution principle
- » Single responsibility principle
- » Law of Demeter

*aka* ***SOLID design principles***

**S**ingle Responsibility Principle.
**O**pen/Closed Principle.
**L**iskov Substitution Principle.
**I**nterface Segregation Principle.
**D**ependency Inversion Principle.

# Single Responsibility Principle

Do one thing only...

# Single Responsibility Principle

† It states that a class or module should have one, and only one, reason to change.

- ♠ An entity (an object, a class) should have only a single responsibility.
- ♠ A responsibility is in fact a reason to change.
- ♠ An entity should have only single reason to change.

**Classes Should Be Small!**

# Single Responsibility Principle

```
public  class  Student {

      ...

      public void readStudent(int  id)  {

           // read student data from  a  database

      ...

      }

      public String printHTML() {

           // print student data in a nice HTML format

      ...

      }

}
```

Example: 1




Example: 2

# Single Responsibility Principle

- Student class has two responsibilities: reading from a database and printing HTML.
- Two reasons for change: if a table is modified and if another HTML format is needed.
- We need to separate responsibilities: reading and printing.
- This is also very often called: *separation of concerns.*

# Single Responsibility Principle

```
public  class  Student {
    ...
    public void readStudent(int  id)  {
       // read student data from  a  database
       ...
    }
}

public class  StudentPrinter  {
    ...
    public String  printStudent(Student  student) {
        //print in a  nice HTML  format
      ...
    }
}
```

Example: 1

```
class SalesOrder{

        void salesOrder()
        {
                // Get information about sales order
        }

}
```

```
class SaveOrders
{
        void saveOrder()
        {
                // Save the order in DB
        }
}
```

```
class ExportOrder
{
        void exportOrderInXML()
        {
                // Export the sales order in XML
        }

        void exportOrderInCSV()
        {
                // Export the sales order in CSV
        }
}
```

Example: 2

# Open-Closed Principle

Open for extension, closed for modification

# Open-Closed Principle: Example

## Online publication system

An online publication system allows users to search in publications by the ***year of publication***. Search functionality is supported by a search filter that filters publications by comparing their publication year with a user query.

# Open-Closed Principle: Example

```
public class Filter {

   public List filterOnYearOfPublication(Collection pubs, int year) {

   List<Publication> filtered = new ArrayList<Publication>();

   for (Publication pub : pubs) {

    if (pub.getYearOfPublication() == year) {

        filtered.add(pub);

        }

    }

    return filtered;

   }

}
```

Online publication system

- Extend the search filter to also filter by the title.

# Open-Closed Principle

- This is the cornerstone principle of OOD; all software changes during its life cycle.
- Programmers face ever changing functional requirements.
- But need to design software that is stable (i.e. does not change).
- Software entities should be open for extension, but closed for modification by B. Meyer.

# Open-Closed Principle

In OO terms

- Classes should be open for extension, but closed for modification.
- In other words you should be able to extend a class without modifying it.
  1. Open for extension: behavior of classes can be extended to satisfy changing requirements.
  2. Closed for modification: classes themselves are not allowed to change.

# Open-Closed Principle

- At first this seems contradictory
- If you need a modified behavior of a class just change that class.
- This is however wrong.
  - The answer here is abstraction.
- In OO it is possible to create abstractions with fixed design but limitless behaviors.

# Open-Closed Principle

- Abstractions through inheritance - changed behavior through polymorphism.
- *Inheritance*: abstract  base classes or interfaces.
- You must decide early on in the design which parts of the system will be expanded later and which will stay fixed.
- The design is extended by adding new code and classes by inheriting existing base classes.
- Modifying the existing classes is not needed.
- Completed and tested code is declared closed, and it is never modified.

# Open-Closed Principle: Example

```
public class Filter {
  public Vector filterPublication(Collection pubs,
     Selector selector) {
  List<Publication> filtered = new ArrayList<Publication>();
  for (Publication pub : pubs) {
         if (selector.isPubSelected(pub)) {
            filtered.add(pub);
         }
  }
      return filtered;
   }
}

public interface Selector {

  public boolean isPubSelected(Publication pub);

}
```

# Open-Closed Principle: Example

```
public  class  YearSelector implements Selector {
   private final int year;
   public YearSelector(int  year) {
      this.year = year;
    }
    public boolean isPubSelected(Publication pub) {
       if (pub.getYearOfPublication() == year) {
           return true;
               }
       return false;
     }
}
```

# Open-Closed Principle: Example

```
public class TitleSelector implements Selector {
    private final  String title;
    public TitleSelector(String  title) {
       this.title = title;
    }
    public boolean isPubSelected(Publication  pub) {
       if (pub.getTitleOfPublication().equals(title)) {
          return true;
       }
    return false;
    }
}
```

# Open-Closed Principle: Example

- Now filtering is open for extension, i.e. adding filters for author, pub type.
- Still filtering is closed for modification - no need to modify the Filter class.
- With the class being open for extension, it is *highly reusable* and can be used to satisfy a ever *changing list of requirements.*
- The class is extremely maintainable, as it never needs to change.

# Liskov Substitution Principle

Preconditions, Post conditions & Invariants

# Design by contract

- Developed by B. Meyer.
- It is a contract between a class and its clients (other classes that use it).
- Specifies rules that need to be satisfied by the class and by the clients.
  - i.e., an expression that evaluates to true.
- Preconditions and post conditions for methods , invariants for the class.

*Example: Stack class*

- Precondition for `push(x)` method: the stack is non-full
- Post condition for `push(x)` method: x is on the top of the stack
- Invariants: stack size is non-negative, count is less than `max_size`, …

※ *In Java: assert keyword to check if rules are satisfied: Testing*

# Liskov Substitution Principle

- Developed by Barbara Liskov in 1987
- It is subtyping relation *(subtyping refers to the ability of a subtype to be used in place of its supertype without altering the correctness of the program. )*
- It is aka ***strong behavioral subtyping (****means that a subtype must not only inherit the methods of its supertype but also maintain the same expected behaviors and adhere to the contracts of the supertype, without introducing unexpected changes or violating its behavior****.)***
- Closely related with Design by Contract.
- Depends on preconditions and post conditions.

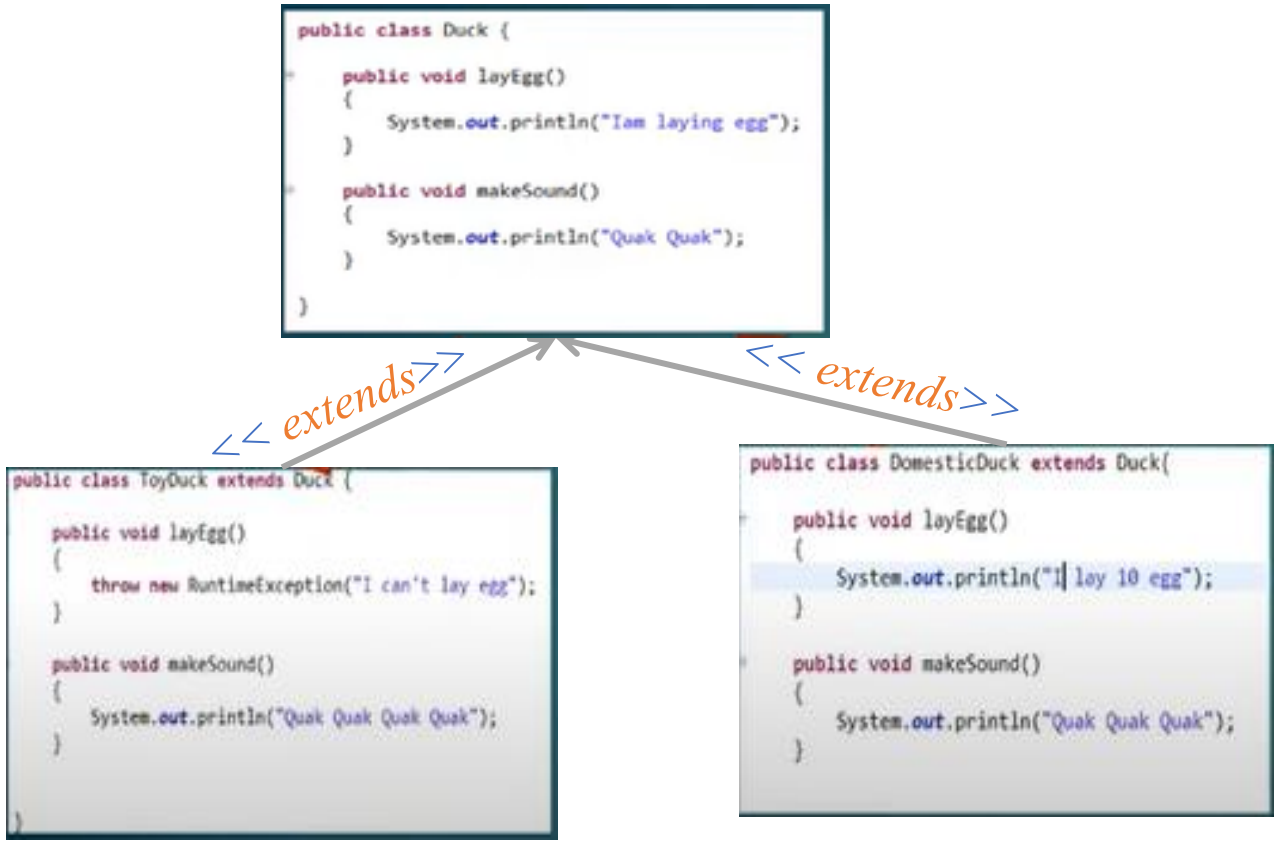
LSP Definition

- It defines that the objects reference of superclass shall be replaceable with objects of its subclass without breaking the application.
- If for each object  of type S there is an object  of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when  is substituted for  then S is a subtype of T.

- What does it mean?

⇒ Anywhere you specify a base type, you should be able to use an instance of a derived type.

# Liskov Substitution Principle



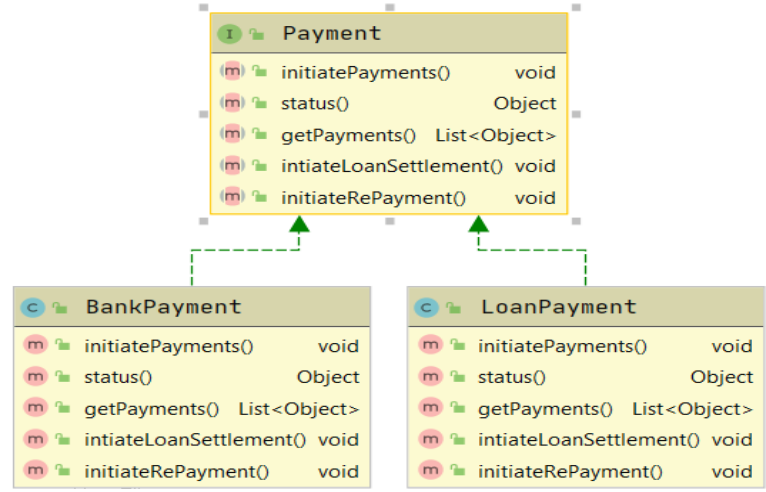

» Base classes need to know about their subclasses.
» Clients of base classes need to know about the sub classes.

# Interface Segregation Principle

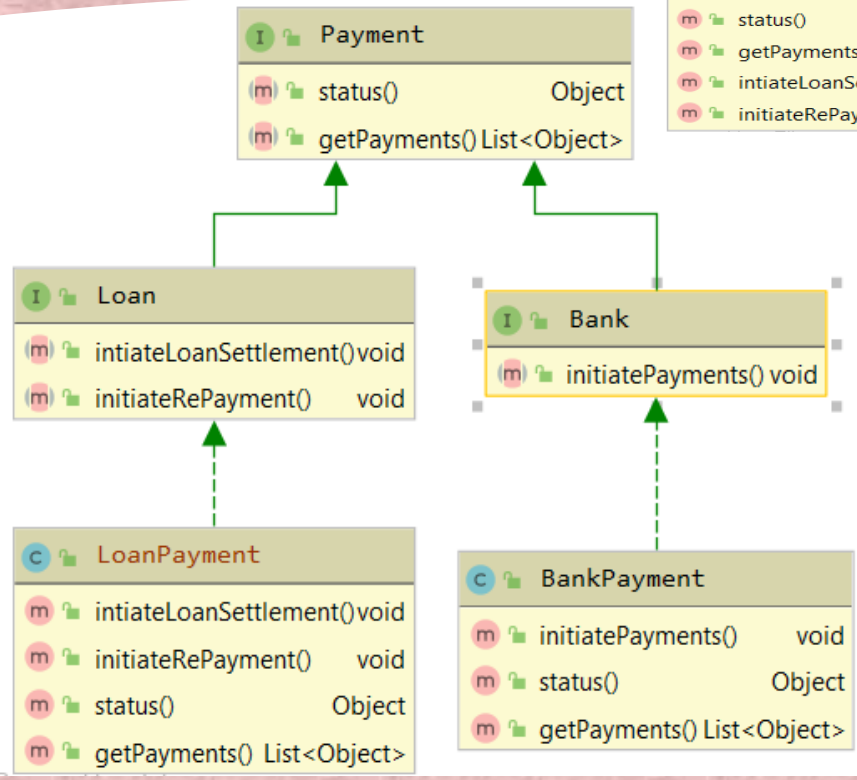
# Interface Segregation Principle

This principle was first defined by Robert C. Martin as: "*Clients should not be forced to depend upon interfaces that they do not use*".

The goal of this principle is *to reduce the side effects of using larger interfaces by breaking application interfaces into smaller ones.*



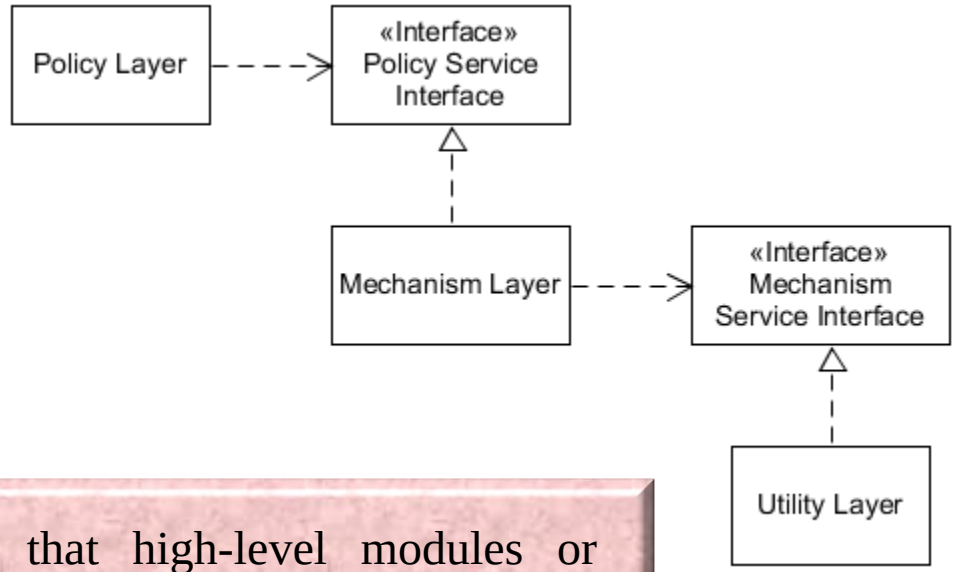

ISP suggests breaking down large, monolithic interfaces into smaller, more specific ones. In short, it encourages interfaces to have only the methods that are relevant to the implementing class, preventing classes from being forced to implement methods they don't need.



# Dependency inversion Principle

# Dependency inversion Principle

- It states that high-level modules should depend on abstractions rather than concrete implementations. This helps decouple the high-level and low-level modules, making it easier to change the low-level ones without affecting the high-level ones.




- Generally, it emphasizes that high-level modules or classes should not depend on low-level ones but both should depend on abstractions (interfaces or abstract classes).
- In short, DIP encourages the decoupling of components by introducing interfaces or abstract classes to mediate interactions, promoting flexibility and maintainability in software design.

# Law of Demeter

Which methods to call...

# Law of Demeter

- A method of an object should only invoke methods of:
  - Itself
  - It's parameters
  - Objects it creates
  - It's members
- Methods should not invoke methods of other objects' members.
- It focus on *reduces dependencies* and helps *build components that are loose coupled* for code reuse, easier maintenance, and testability.

# Law of Demeter

```
public class  StudentPrinter  {
      ...
      public String printStudent(Student student) {
      ...
      // violation
      String  course = student.getCourse(i).getName();
      }
 }

public  class  Student {

...

public  String  getCourseName(int i) {

  return courses[i].getName();

    }

}
```

# Law of Demeter

- Chaining of method calls is dangerous because you depend on all parts of that chain.
- Provide a method in Student class that gets you names of the courses.
- That method delegates to the Course class.
- If the Course class changes only Student class needs to be updated not StudentPrinter.
- Consequence for composition/delegation: only delegate to own members!

# Architecture Principles

Collection of best practices

# Architecture Principles

- **Reuse**
  - Existing components must be reused where applicable; Instead of developing them from scratch.
  - In practice there is often a trade-off, as the existing component might not exactly match the requirements.
  - Optionally, extend the existing component.
- **Buy rather than build**
  - If a component exist (commercially or open-source) that match the requirements, it should rather be used.
  - Unless there are strategically (e.g. competition) or technical issues.
  - ***Motivation***: higher quality of existing solutions, might provide extra functionality.
  - Often, it is cheaper to use existing solutions instead of developing them.

# Architecture Principles

- **Single point of view**
  - ↳ If there are multiple sources of data (e.g. file-system and database system).
  - ↳ Develop components to make them appear as a single source.

***Motivation***: will lead to more simple components (that use the source), logic to combine the sources is bundled in a single place.

***Reading Assignment !!1***

» Principle of Least Knowledge
» Minimize Large Design Upfront
» Do not Repeat the Functionality
» Prefer Composition over Inheritance while Reusing the Functionality


Q
&
A

# Quiz-2
# Discuss the following with what you learned.

**S**ingle Responsibility Principle.

**O**pen/Closed Principle.

**L**iskov Substitution Principle.

**I**nterface Segregation Principle.

**D**ependency Inversion Principle.

# END

Next: Object-Oriented Design Patterns

# Contents

## Chapter 02: OO Design Patterns

- What are design patterns
- Creational design patterns
- Structural design patterns
- Behavioral design patterns

*" Design Patterns - Elements of Reusable Object-Oriented Software. "* - *Author: Erich Gamma*